

Ocelot3 Part2

Kishore Senthil Kumar

Dec 12, 2025



Agenda

- Background
- RISC-V Summit
- Precise Trap
- Verification and Debugging
- Updated Final Diagram
- Future Roadmap
- Conclusion

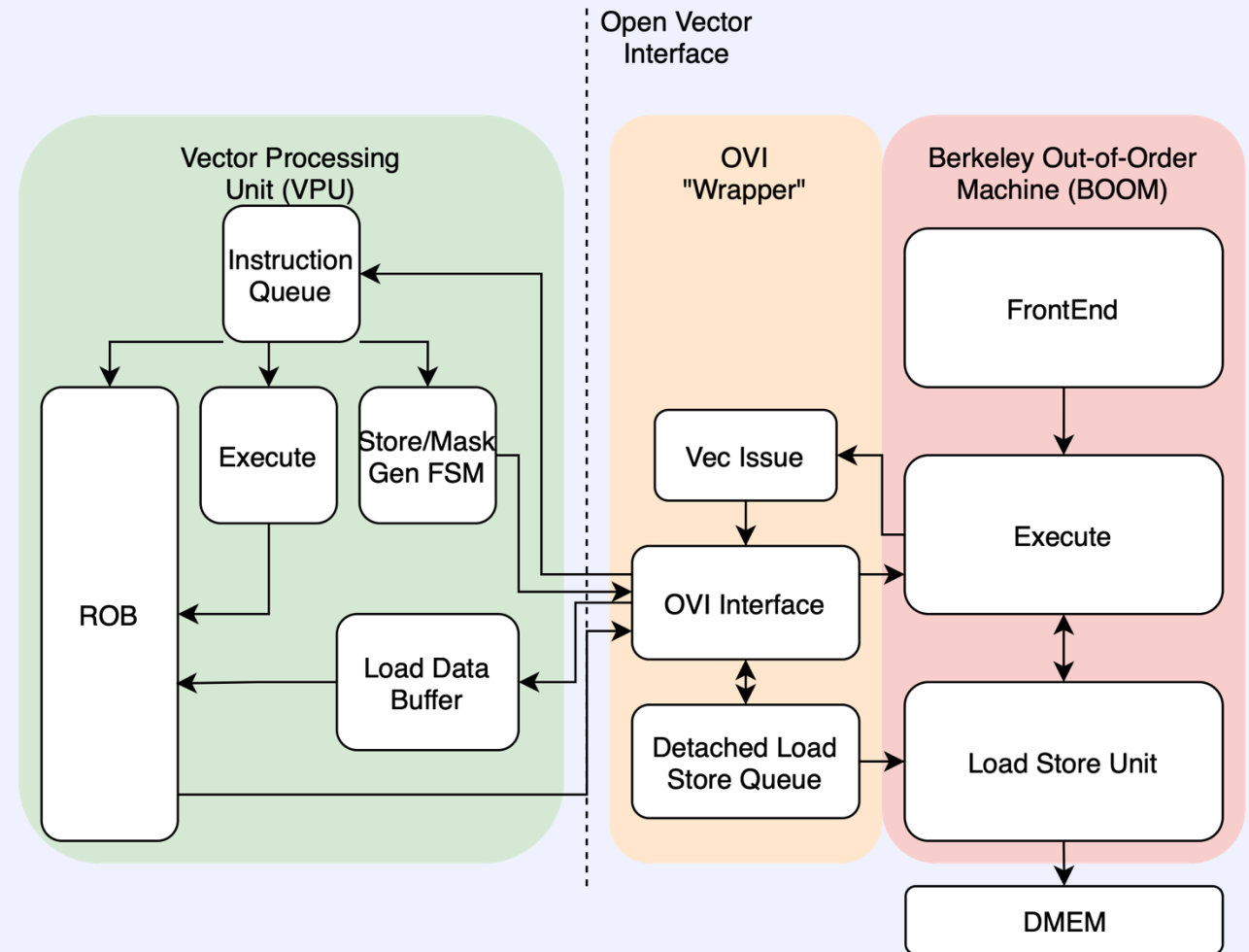


Ocelot 3: Project Background



The Ocelot Project

- Open-Source Project:
<https://github.com/tenstorrent/riscv-ocelot/tree/bobcat>
- A vector extension to the Boom core
- VPU inspired by Baby RISC-V
- VPU connected through OVI protocol
- VPU shares memory interface with CPU
- Implemented Precise Trap Mechanism



Ocelot 3: RISC-V Summit Poster Presentation



RISC-V Summit Poster Presentation

- Increased git clone traffic
- Selected by committee for interview



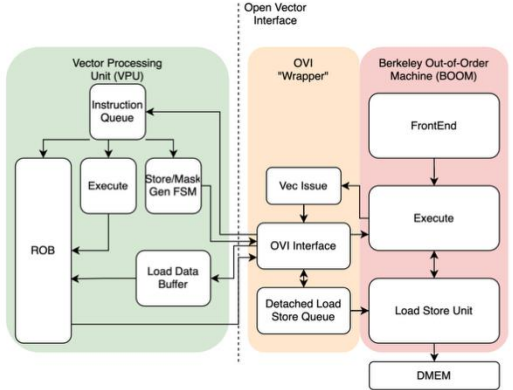
**RISC-V
SUMMIT**
NORTH AMERICA 2025

Ocelot3: Full Vector “V” Extension for BOOM


github.com/tensorrent/riscv-ocelot

Overview

- Vector Extension Support for BOOM
- Full RVV 1.0 Support
- Improvement from **Ocelot2** (added segment load / store, division, sqrt)
- Open Vector Interface (OVI) compatibility
- Integrated with RiscueC Test Generator



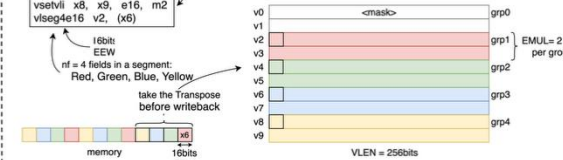
The diagram illustrates the system architecture. On the left, the **Vector Processing Unit (VPU)** contains an **Instruction Queue**, **Execute** block, **Store/Mask Gen FSM**, **ROB** (Reorder Buffer), and **Load Data Buffer**. On the right, the **Berkeley Out-of-Order Machine (BOOM)** includes a **FrontEnd**, **Execute** block, **Load Store Unit**, and **DMEM** (Data Memory). An **OVI "Wrapper"** sits between them, containing a **Vec Issue** block, **OVI Interface**, and **Detached Load Store Queue**. Arrows show the flow of instructions and data between these components.

Feature Highlight: Segment Instructions

SEW = EEW so no widening/narrowing
LMUL = 2 regs / group

```
li x9, 15  
vsetvli x8, x9, e16, m2  
vlsseg4e16 v2, (x6)
```

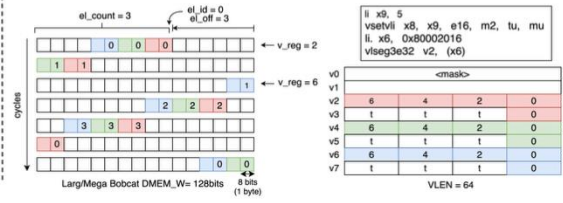
16bit EEW
nf = 4 fields in a segment: Red, Green, Blue, Yellow
take the Transpose before writeback



The diagram shows a memory segment of 16 bits (EEW) divided into four 4-bit fields (Red, Green, Blue, Yellow). These fields are transposed and then written back to the register file. The register file is organized into groups (grp0 to grp4) with a VLEN of 256 bits.

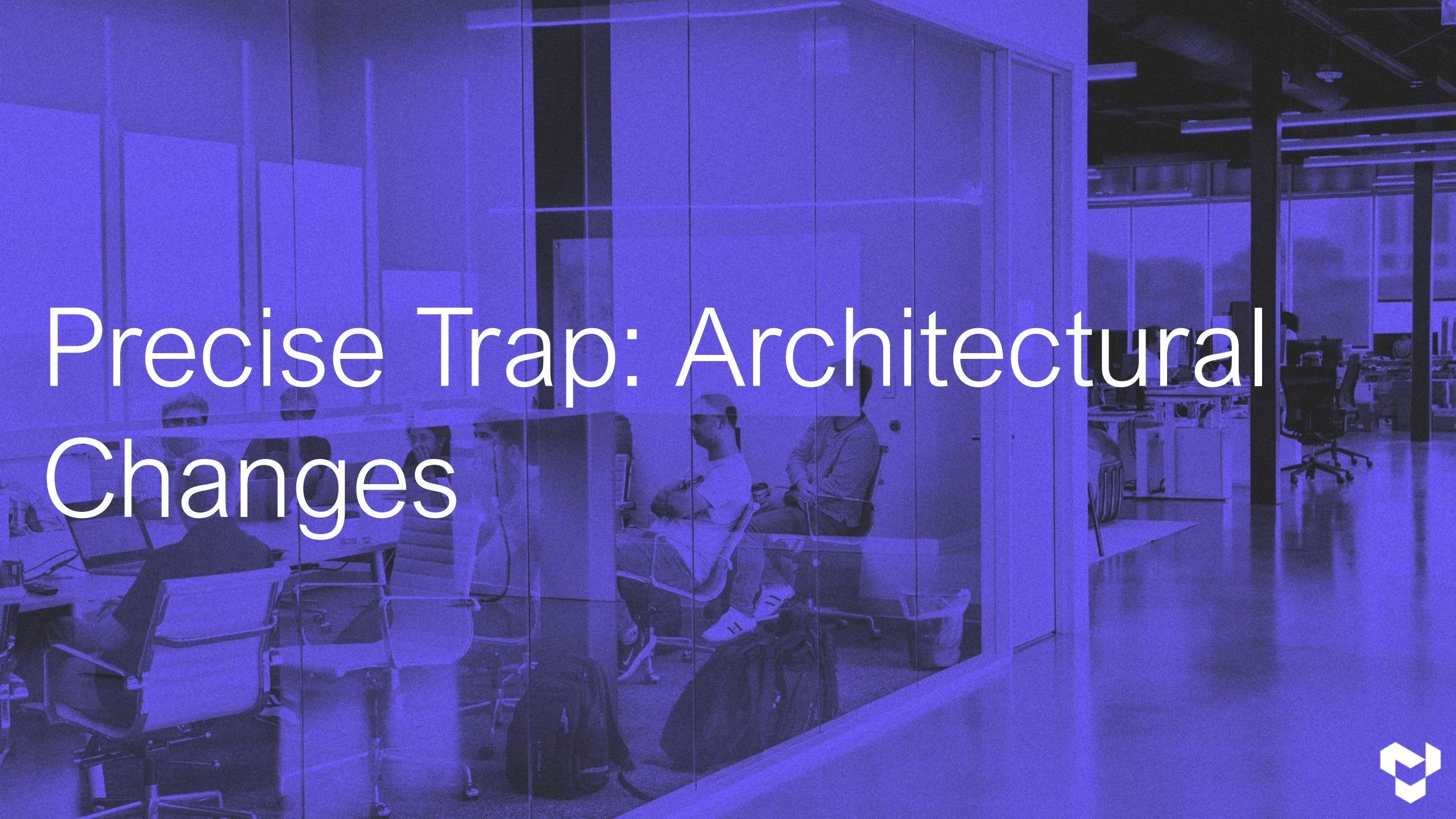
Example:

Number of Segments = 3, EEW = 16, EMUL = 2, VL = 5, VD = v2



The example shows a sequence of instructions: `li x0, 5`, `vsetvli x8, x9, e16, m2, tu, mu`, `li x6, 0x80002016`, and `vlsseg3e32 v2, (x6)`. The diagram illustrates the state of the register file and the segment load operation, showing the transposition of the segment data into the register file.

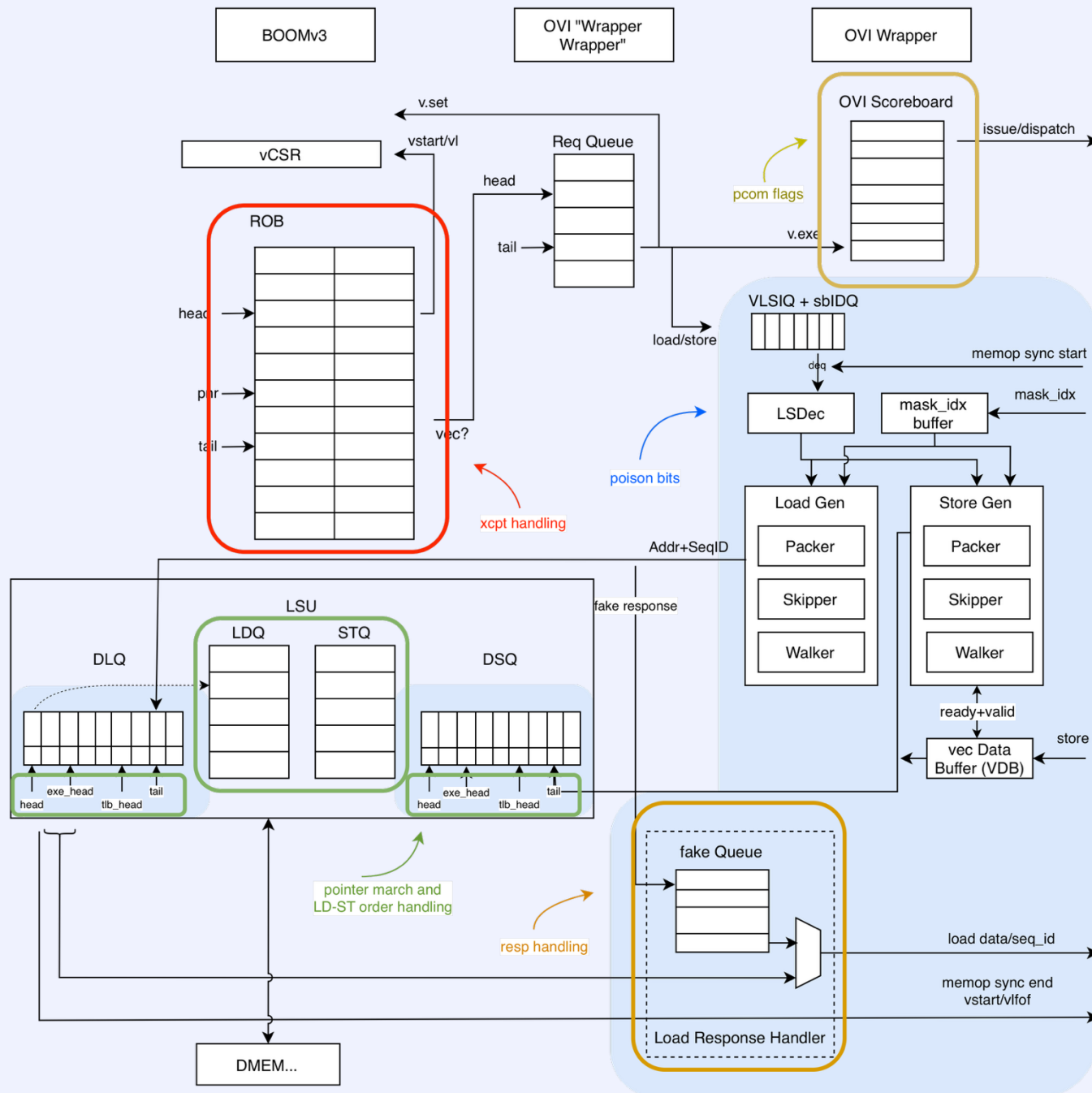




Precise Trap: Architectural Changes

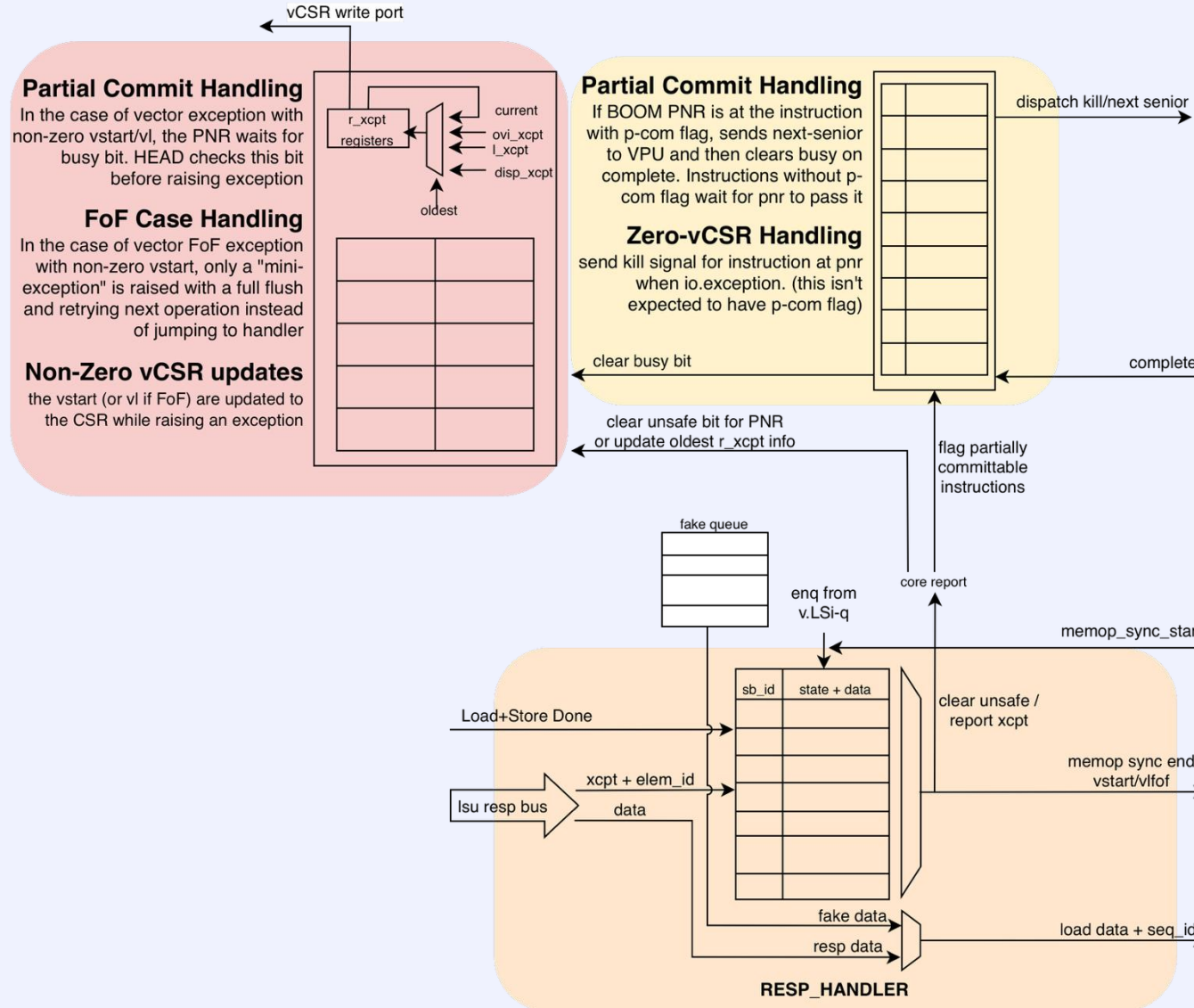


Overview of Changes in OVI and BOOM Core

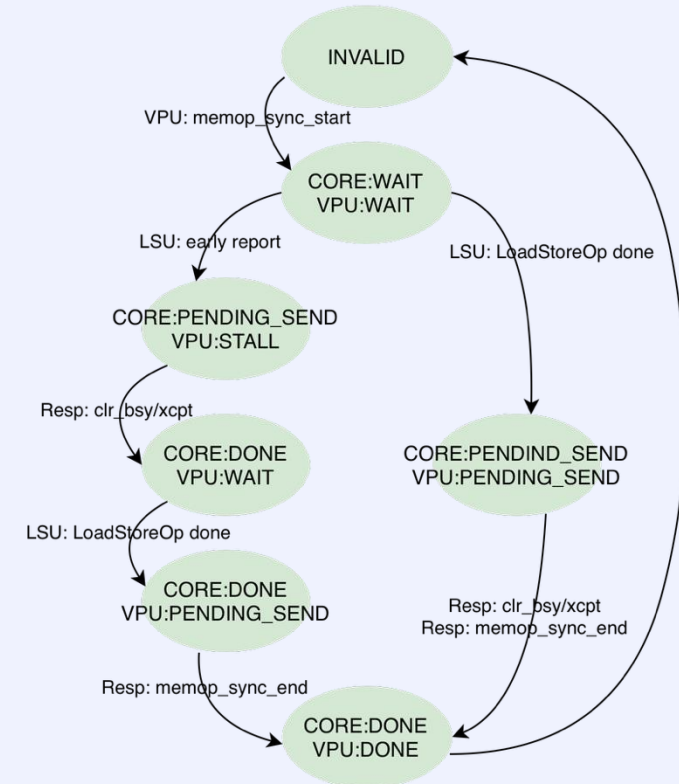


- Speculative state throughout OVI pipeline
- (D)LSU entries and pointer logic
- Response Handler logic
- ROB vCSR and clrbsy ports
- vStart masking in VPU

Response handling in OVI

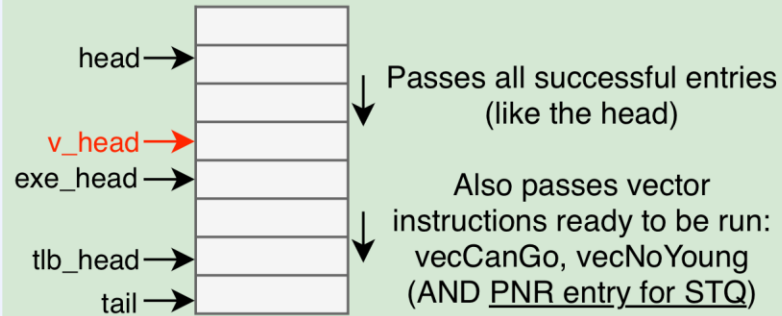


Resp Handler FSM



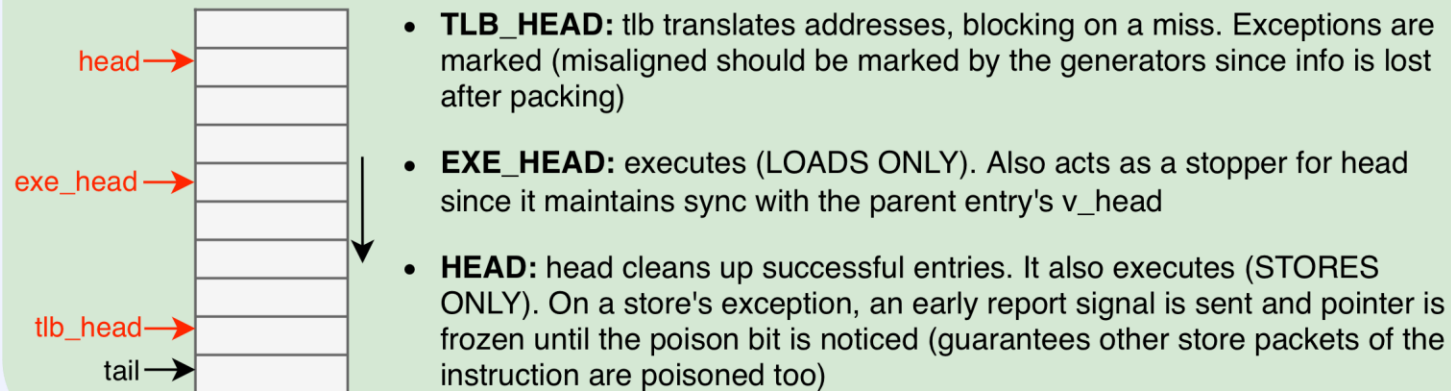
LSU changes

LDQ / STQ



1. Being or becoming the memop entry sets the vecCanGo/vecNoYoung bits
2. This lets v_head pass the parent entry for vector operations
3. Detached queues wait until v_head passes their parent entry to execute
4. Once complete, the last detached entry will mark the parent successful
5. Succeeded entries allow the next instruction to proceed (not vec or set bits)
6. Upon poison, parent entry is never checked since it won't exist (for v_head or success bit purposes)

DLQ / DSQ



Speculative State

Architectural state must be checkpointed for vector operations in the LSU as well. Beyond simply keeping the b_masks fresh and maintaining poison bits, we also need to rewind the dsq/dlq ptrs and the v_head of the stq/ldq. on mispredicts and exceptions

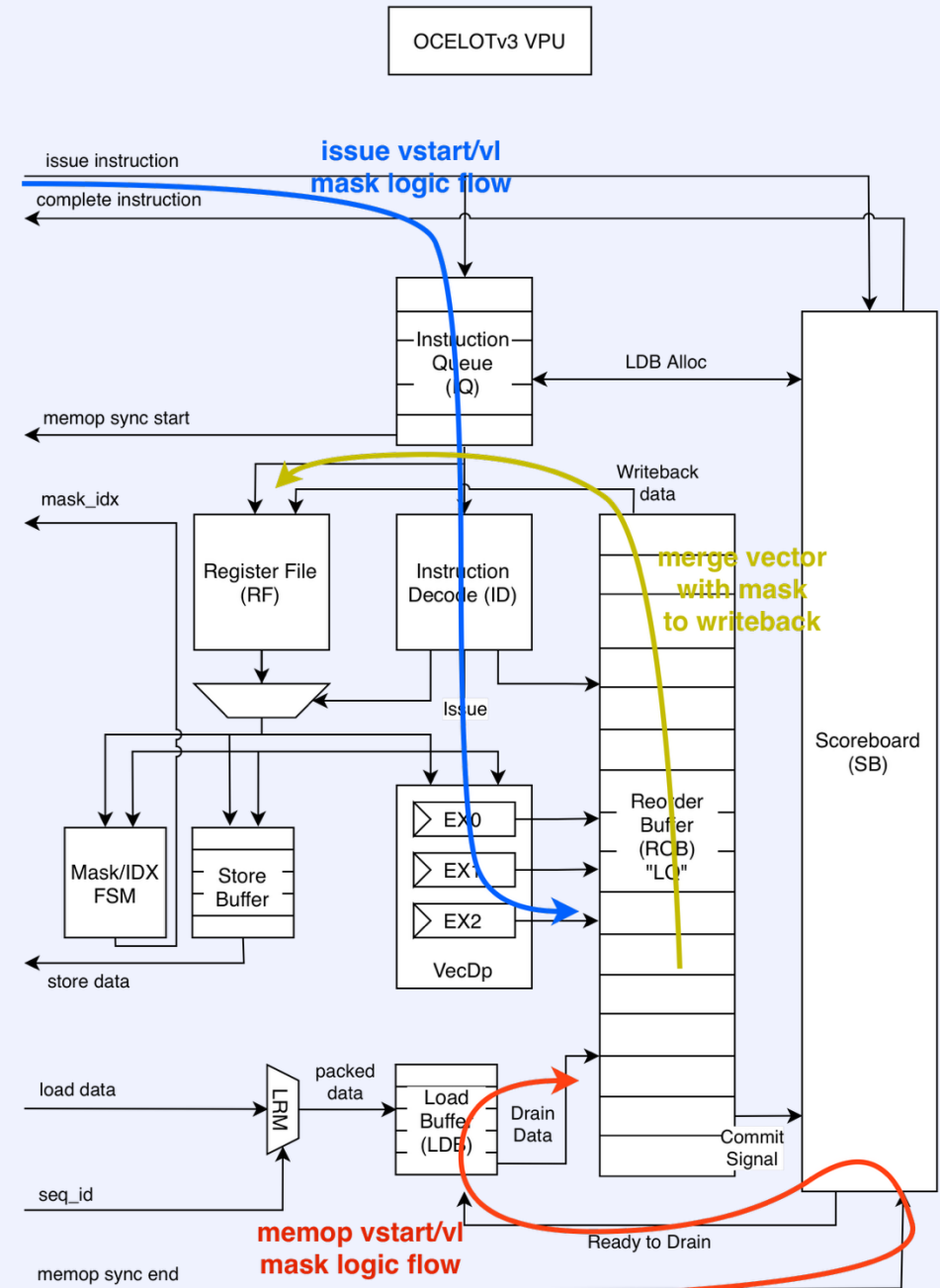
Execution State

Loads can speculatively execute whenever possible. No problem for vector!

Stores typically execute after retiring. However, for vector, we need to start executing to guarantee safety. So they execute when they are the oldest memory instruction when the pnr is pointing to it (not guaranteed safe by pnr but it is the only instruction stopping from the pnr proceeding)

VPU vstart/vl masking logic

- Instruction masking must be done when instruction is issued with any vstart value (similar to vl logic done in tt_vec.sv)
- Partial writeback requires another layer of vstart masking. This is sent from the OVI memop bus
- Both masking logic are considered before merging in the “LQ” (ROB) of the VPU



Verification and Debugging



Testcase for precise trap

- No interface to thoroughly test the trap but checked architectural state during jump

```
vmv.v.v x8, x1
// config
li x9, 5 // vl = 5
vsetvli x8, x9, e16, m2, tu, mu // LMUL = 2, SEW = 16, VLEN = 256

// address 1
li x2, 0
li x3, 0 // success
call set_v4_elem_value

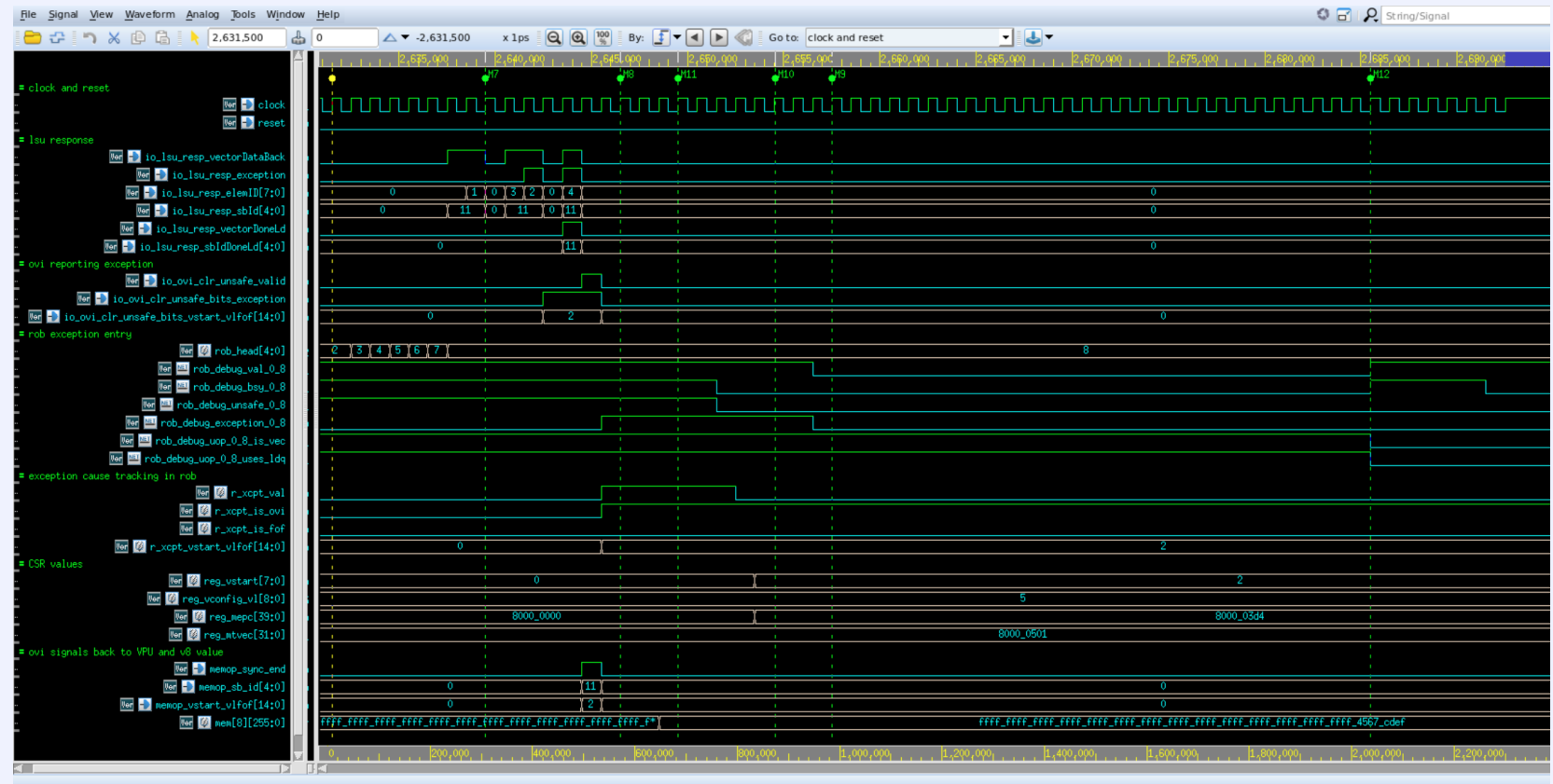
// address 2
li x2, 1
li x3, 4 // success
call set_v4_elem_value

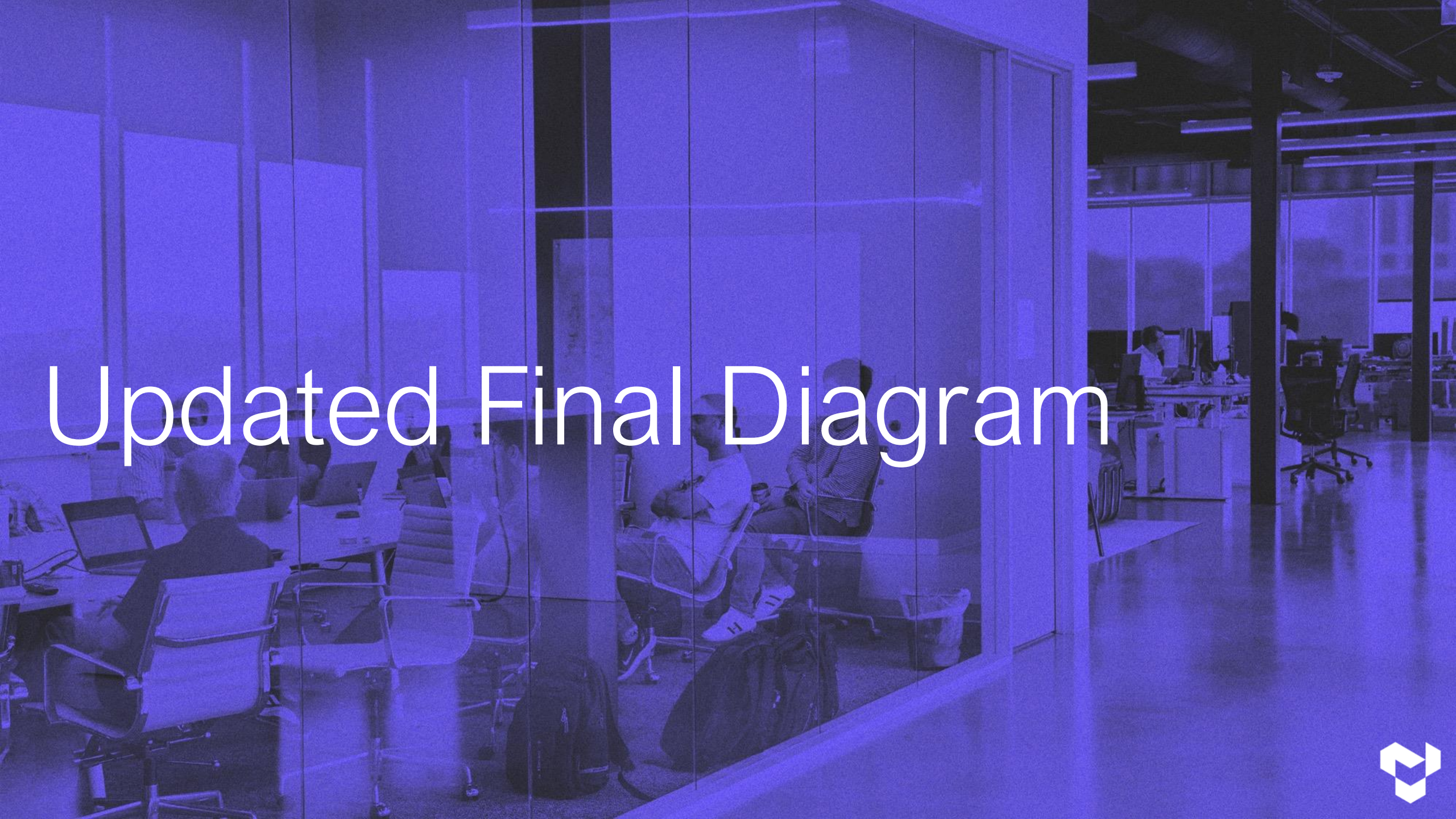
// address 3
li x2, 2
li x3, -1 // UNALIGNED
call set_v4_elem_value

// address 4
li x2, 3
li x3, 4 // not run
call set_v4_elem_value

// address 5
li x2, 4
li x3, -1 // not run
call set_v4_elem_value

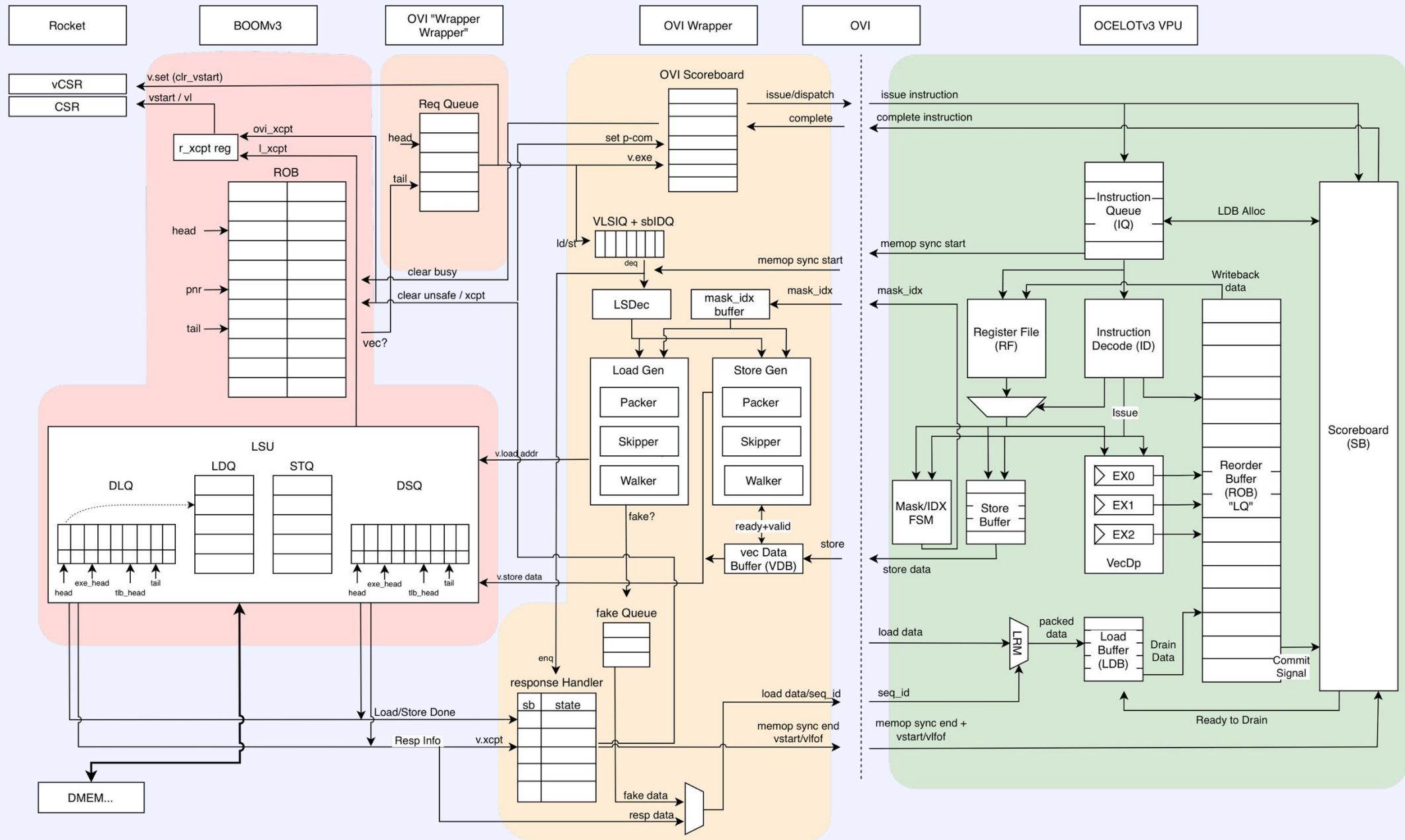
// load test
li x6, 0x80002010
vloxei16.v v8, (x6), v4 // TRAP on idx2
csrr t2, vstart
csrr t3, vl
```





Updated Final Diagram







Future Roadmap: Ocelot 4??



Possible Improvements

- vCSR renaming
- Vector Load-Store Forwarding
- Vector-Scalar Forwarding
- VPU full OoO execution (vector renaming)
- Updating BOOM repository
- Officially verify RVV "V" compliance

Thank You

Special thanks:

- DJ
- Kuan-Yu Chen

